

电子与信息学报
Journal of Electronics & Information Technology
ISSN 1009-5896, CN 11-4494/TN

《电子与信息学报》网络首发论文

题目：基于图和代码切片的可解释性漏洞检测方法
作者：高文超，索建华，张傲
收稿日期：2025-05-06
网络首发日期：2025-12-29
引用格式：高文超，索建华，张傲. 基于图和代码切片的可解释性漏洞检测方法[J/OL]. 电子与信息学报. <https://link.cnki.net/urlid/11.4494.TN.20251226.1120.004>



网络首发：在编辑部工作流程中，稿件从录用到出版要经历录用定稿、排版定稿、整期汇编定稿等阶段。录用定稿指内容已经确定，且通过同行评议、主编终审同意刊用的稿件。排版定稿指录用定稿按照期刊特定版式（包括网络呈现版式）排版后的稿件，可暂不确定出版年、卷、期和页码。整期汇编定稿指出版年、卷、期、页码均已确定的印刷或数字出版的整期汇编稿件。录用定稿网络首发稿件内容必须符合《出版管理条例》和《期刊出版管理规定》的有关规定；学术研究成果具有创新性、科学性和先进性，符合编辑部对刊文的录用要求，不存在学术不端行为及其他侵权行为；稿件内容应基本符合国家有关书刊编辑、出版的技术标准，正确使用和统一规范语言文字、符号、数字、外文字母、法定计量单位及地图标注等。为确保录用定稿网络首发的严肃性，录用定稿一经发布，不得修改论文题目、作者、机构名称和学术内容，只可基于编辑规范进行少量文字的修改。

出版确认：纸质期刊编辑部通过与《中国学术期刊（光盘版）》电子杂志社有限公司签约，在《中国学术期刊（网络版）》出版传播平台上创办与纸质期刊内容一致的网络版，以单篇或整期出版形式，在印刷出版之前刊发论文的录用定稿、排版定稿、整期汇编定稿。因为《中国学术期刊（网络版）》是国家新闻出版广电总局批准的网络连续型出版物（ISSN 2096-4188，CN 11-6037/Z），所以签约期刊的网络版上网络首发论文视为正式出版。

基于图和代码切片的可解释性漏洞检测方法

高文超* 索建华 张 傲

(中国矿业大学(北京)人工智能学院 北京 100083)

摘 要：深度学习已被广泛应用于漏洞检测，其主流方法可分为基于代码序列和基于代码图两类：前者易因忽视结构而误报，后者则难以捕获执行顺序。此外，两者普遍缺乏可解释性，难以定位漏洞根源。为此，该文提出一种基于图和代码切片的可解释性漏洞检测方法GSVD。该模型通过门控图卷积网络提取代码多维度图(AST, DDG, CDG)的结构语义，并结合“污点”分析驱动的代码切片与双向长短时记忆网络，精准捕获代码序列特征，实现二者优势互补。同时，引入HITS算法思想，设计VDExplainer解释器，直观揭示了模型的决策过程。实验表明，GSVD在Devign数据集上准确率达64.57%，优于多种基线模型，证明了其在有效检测漏洞的同时，能实现现代码行级的可解释定位。

关键词：漏洞检测；深度学习；图神经网络；代码切片；可解释性

文献标识码：A

DOI: 10.11999/JEIT250363

文章编号：1009-5896(2025)12-0001-10

CSTR: 32379.14.JEIT250363

1 引言

随着社会的不断发展，软件程序为满足用户在自动化、安全性、功能扩展和跨平台兼容性等方面的需求而日趋复杂。《2024中国软件供应链安全分析报告》显示，国内企业的软件项目全部使用了开源软件，平均每个项目包含166个开源组件，其中超过八成存在高危漏洞，平均每个项目检测出83个已知漏洞。这表明，网络空间安全问题日益严峻，已成为维护国家安全的重要核心。

软件漏洞检测是保障网络空间安全的关键挑战。早期研究依赖静态分析^[1]、动态分析^[2]和符号执行等传统方法，但这些方法高度依赖专家定义的漏洞规则，难以应对新型漏洞和庞大的代码规模，导致效率低、召回率差等问题。因此，学术界和工业界迫切需要能自动从代码中学习漏洞特征的端到端方法。

受益于自然语言处理领域的成功，早期的深度学习漏洞检测研究大多将源代码视为一种特殊的文本序列。典型代表工作是Li等人^[3]提出的VulDeePecker，他们创新性地提出了“code gadget”概念，并使用双向长短时记忆网络(Bidirectional Long Short-Term Memory Network, BiLSTM)进行训练，显著提升了检测效果。然而，VulDeePecker在生成gadget时仅考虑了数据依赖，存在明显局限。为解决此问题，Zou等人^[4]在此基础上同时引入了控制依赖，提出了 μ VulDeePecker，并使用building-block BiLSTM融合不同特征。另一代

表性工作SySeVR^[5]则通过分析抽象语法树(Abstract Syntax Tree, AST)提取API调用、指针使用等关键语法元素，并利用双向门控循环单元(Bidirectional Gated Recurrent Unit, Bi-GRU)学习漏洞模式。然而，无论VulDeePecker的迭代还是SySeVR的改进，它们都未能摆脱将结构化代码“扁平化”为一维序列的根本性束缚。由于忽视程序结构，此类方法虽能捕获语义信息，但可解释性较弱，且在复杂漏洞的精细化定位上存在根本性局限。

为了克服序列模型在结构建模上的局限，研究者开始尝试以图结构对源代码进行建模。代表性工作包括Zhou等人^[6]提出的Devign，其基于抽象语法树AST、控制依赖图(Control Dependency Graph, CDG)和数据依赖图(Data Dependency Graph, DDG)构建复合代码图，并利用带有卷积模块的门控图神经网络(Gated Graph Neural Network, GGNN)进行图级分类，在多个真实数据集上验证了基于图的漏洞检测方法的有效性。随后，Feng等人^[7]通过对多种图表示、不同节点嵌入方式以及多个图神经网络组合进行系统性实验，进一步确认了基于图进行深度学习漏洞检测的性能优势。Nguyen等人^[8]提出的ReGVD模型在此基础上优化了图构建与残差连接机制，显著提升了在CodeXGLUE等基准上的检测精度。另外，Chakraborty等人^[9]提出了Reveal漏洞检测方法，利用代码属性图(Code Property Graph, CPG)整合语法、语义与数据流信息，并针对真实场景中存在的数据不平衡与特征无关性问题进行了改进，在函数级漏洞检测任务中取得了较好效果。近期，Contract-guardian^[10]则将图结构作为特征工程来源，再结合高效的传统机器学习模型用

收稿日期：2025-05-06；改回日期：2025-11-13

*通信作者：高文超 gaowc@cumtb.edu.cn

于检测智能合约漏洞。然而,基于图的方法通常只能在函数或代码片段级别完成整体检测,难以精确定位具体的漏洞语句;此外,多数GNN模型仍表现为“黑盒”结构,其决策过程缺乏透明性和可解释性。

近年来,以Transformer架构为基础的大语言模型(Large Language Model, LLM)因其强大的代码理解和生成能力,为漏洞检测带来了新的范式。研究者探索了多种应用路径:从直接应用于业界的混合专家(Mixture of Experts, MoE)代码大语言模型如DeepSeek-Coder^[11,12],到利用大量网络安全文本微调基于Transformer的双向编码器(Bidirectional Encoder Representations from Transformers, BERT)得到一个网络安全领域专有的语言模型SecureBERT^[13],再到LLM4Vul^[14]中提出了一个用于解耦合增强大语言模型漏洞推理的统一评估框架。但LLM应用于安全领域仍面临严峻挑战:其一,高昂的计算和部署成本;其二,模型的“幻觉”问题在安全领域是致命的,可能谎报漏洞或提供错误修复^[15,16];其三,LLM本质上是概率模型,对程序的结构化逻辑理解有限,在需要高可靠性的场景中仍存不足。

基于上述问题,本文提出了一种基于图和代码切片的可解释性漏洞检测方法(interpretable Vulnerability Detection method based on Graph and code Slicing, GSVD)。本文的主要贡献如下:

(1) 提出了一种图与序列深度融合的表征框

架,以克服单一模型的表征局限。本文通过带有自注意力池化层的门控图卷积网络(Gated Graph Convolutional Network, GGCN)从代码的复合图结构(AST, DDG, CDG)中学习其深层结构特征,同时创新性地利用基于“污点”分析的代码切片算法,精准聚焦并提取与漏洞高度相关的序列语义特征,实现了二者的优势互补。

(2) 针对GNN模型决策过程不透明,以及现有解释器以边为中心、难以进行有效节点定位的问题,本文设计了可解释器VDExplainer。本方法创新性地引入HITS算法思想,通过迭代计算节点的权威值与枢纽值来直接评估其重要性,为漏洞预测提供了节点级的归因解释,有效提升了模型的透明度与可信度。

2 GSVD模型设计

2.1 模型整体架构

模型整体架构如图1所示。

本文漏洞检测模型主要流程如下。

(1) 数据预处理:对源代码数据进行预处理操作。生成代码图结构,同时利用代码切片算法生成代码切片。

(2) 特征提取:完成代码图结构中节点特征和邻接矩阵的提取以及代码序列特征向量的提取。

(3) 模型训练:使用漏洞检测模型进行训练和特征融合,漏洞检测模型由GGCN, BiLSTM和GRU模块组成,并通过输出层完成代码分类任务。

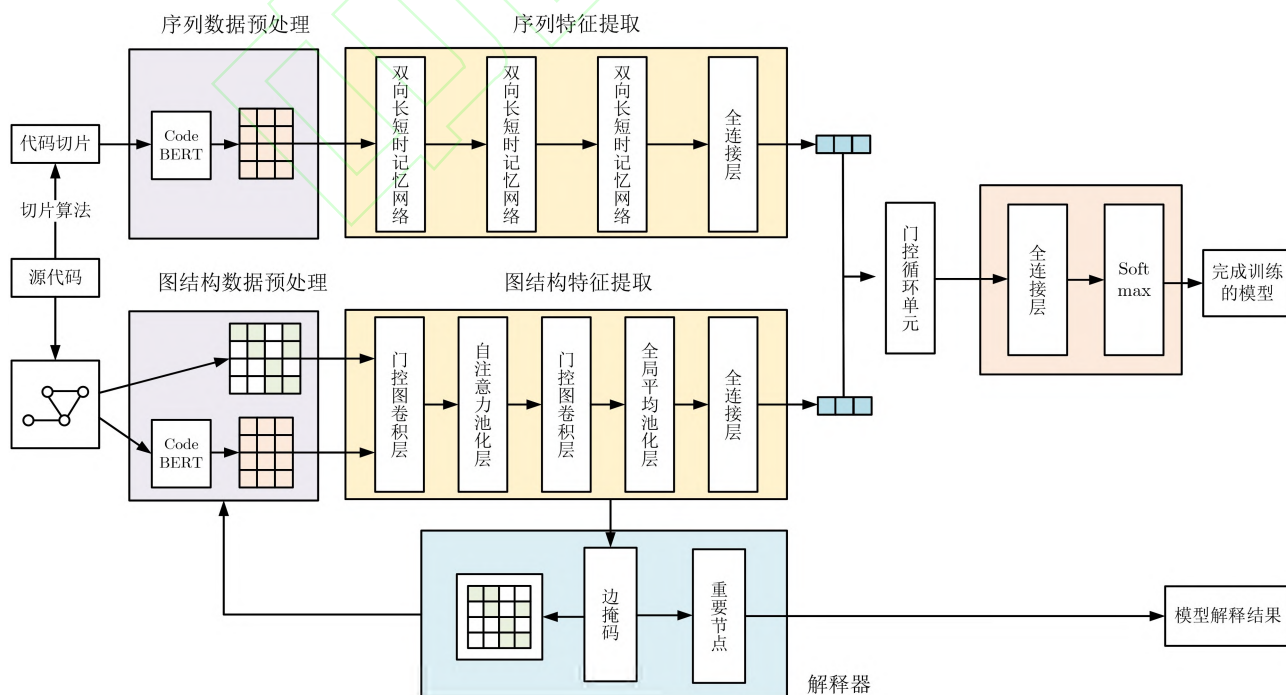


图1 GSVD模型整体架构

(4) 模型解释：使用解释器VDExplainer根据检测影响生成图中的节点重要性排序，对模型检测结果进行解释。

2.2 代码图结构特征提取

2.2.1 图结构数据预处理

(1) 规范化处理

源代码数据中包含丰富的语义信息，但也存在一些干扰内容，例如注释、变量名、函数名以及某些字面值信息等，这些无关信息会干扰深度学习模型的训练和测试。因此，本文在建立代码图结构之前，首先进行了规范化处理。规范化处理不仅能减少漏洞检测中的无关信息干扰，还能降低后续任务的资源消耗。具体的规范化要求如下：

(a) 去除所有注释内容。

(b) 替换变量名为VAR_1, VAR_2, ..., VAR_n的形式。

(c) 替换用户自定义的函数名为FUNC_1, FUNC_2, ..., FUNC_n的形式。

(d) 替换整型、浮点型数据字面值为INT_1, INT_2, ..., INT_n和FLOAT_1, FLOAT_2, ..., FLOAT_n的形式。

(2) 图结构生成

为了全面地检测漏洞，本文基于AST, DDG和CDG这3种图结构构建漏洞检测模型的图特征。同时，为了减少后续任务的训练消耗，本文将生成的3种图进行融合，以此作为代码的图结构表示。在具体实现中，本文首先利用静态代码解析工具Joern提取源代码中的依赖关系和AST。随后，以AST为基础，将DDG和CDG中的数据流和控制流

信息以边的形式添加到AST中，完成代码图结构的构建。以展示代码为例，其生成的代码图表征如图2所示。

2.2.2 图结构特征提取

为使代码图表征可用于训练，本文对其进行特征提取，分别处理节点与边数据。节点特征由两部分拼接而成。(1) 节点内容特征：使用一个预训练好的CodeBERT模型将节点代码内容编码为特征向量，并提取其[CLS]标记对应的向量。(2) 节点类型特征：统计图中所有节点类型(如ShiftExpression, Statement, ArgumentList, IdentifierDecl等)，对其进行分类并通过one-hot编码转换为向量。边数据(即图的拓扑结构)则通过邻接矩阵进行表征，该矩阵直接反映节点间的连接关系(若节点*i*到*j*存在边，则矩阵中对应值为1)，作为图神经网络进行信息传播和聚合的载体，以实现对图数据的有效学习。

在获取代码图特征后，本文利用GGCN对节点特征进行训练。经过GGCN层处理后，节点的特征表示已融合了相邻节点及边的信息，可用于后续的图向量聚合与漏洞检测。值得注意的是，为避免遗漏关键节点并确保模型能应用于更广泛的数据集，本文未通过图剪裁或筛选来限定代码图的节点数量。此设计虽最大限度地保留了图的完整信息，但也相应地增加了计算开销。

为更好地锁定与漏洞相关的节点，并解决代码图结构特征利用不足的问题，本文在GGCN之间引入了自注意力池化层。该层通过计算节点的注意力得分来筛选节点和更新图数据，使模型对关键节点更加敏感。节点的注意力得分计算方式如公式(1)所示

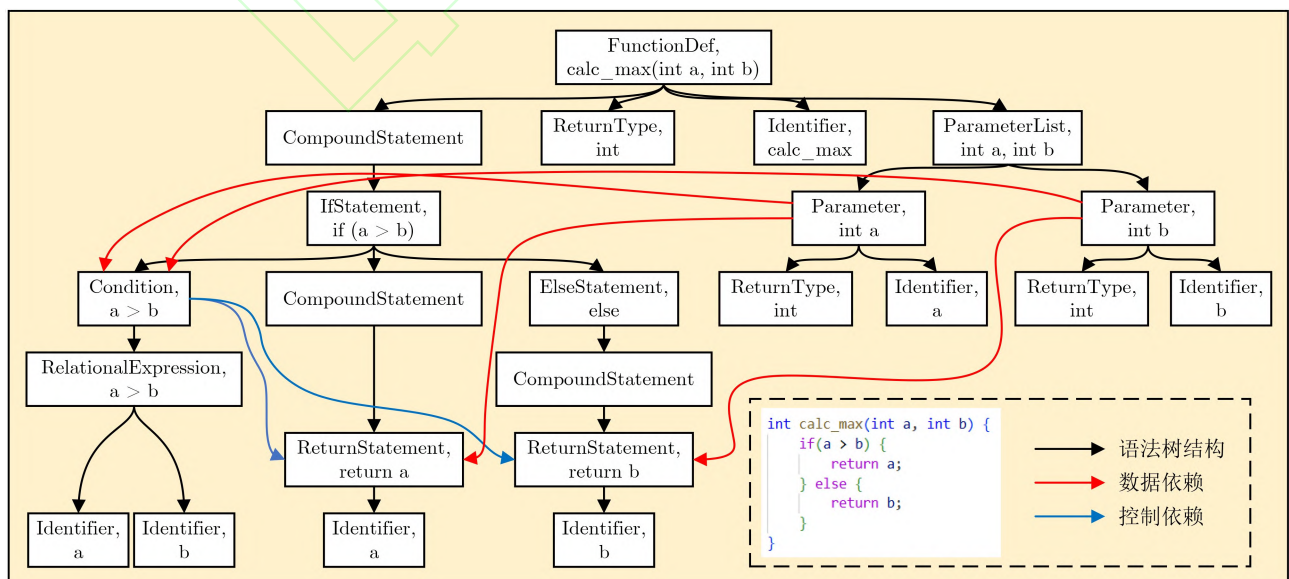


图2 代码图表征

$$Z = \sigma(\text{GGCN}(\mathbf{X}, \mathbf{A})) \quad (1)$$

其中 Z 是注意力得分, σ 表示激活函数, 这里使用Sigmoid函数, 用于将得分限制在0和1之间, $\mathbf{X}$ 表示节点特征矩阵, 其中每一行代表一个节点的特征向量, $\mathbf{A}$ 是图的邻接矩阵。

在获取节点注意力得分后, 通过top-k方法选择节点, 并根据节点筛选比例确定保留的节点数量, 最终更新代码图数据, 具体的计算过程如公式(2)所示

$$\text{idx} = \text{top} - \text{rank}(Z, [kN]) \quad (2)$$

其中idx表示节点索引列表, top - rank表示排序操作, Z 表示各节点注意力得分, k 表示节点筛选比例, N 表示图中节点总数。

最后, 使用一个全局平均池化层对门控图卷积层的输出进行处理, 通过对所有节点的特征向量取平均值, 以此来生成整个图的向量表示。

2.3 代码序列特征提取

2.3.1 代码切片

(1)建立分析结构

抽象语法树作为代码的结构表示, 能够很好地体现代码序列中标记的所在位置。首先通过Jorn生成AST, 随后对AST进行剪枝, 将根节点视为函数定义, 保留中间的语句节点, 并去除代表标记的叶节点。最后, 将DDG和CDG中的依赖关系合并入此AST。在此过程中, 如果原数据或控制依赖指向一个已被移除的叶节点, 该依赖将向上查找, 直至关联到首个保留的父语句节点。

(2)初始化污点

在污点传播过程中, 首先需要识别和标记所有潜在的不安全输入源。当程序接收到来自外部的输入时, 将这些数据标记为“污点”。由于数据集的数据是函数级的代码程序, 污点数据即为函数外部输入的数据。在这里, 主要考虑将以下4种情况的数据作为污点。

(a)函数参数: 函数调用时由外部传入。

(b)全局变量: 函数外部定义但在函数中使用。

(c)文件读取: 函数通过文件读入外部数据。

(d)标准输入: 通过scanf, fgets等函数从控制台获取用户输入。

本文在识别污点后将其整理为污点序列以供切片分析。对于(a)和(b), 若变量未在函数内定义, 则视为外部输入并直接加入污点序列; 而(c)和(d)需在传播过程中确认是否接收到外部数据后再标记为污点。涉及的外部输入函数主要包括scanf, fscanf, sscanf, gets, fgets, getc, fgetc, getchar和read。

(3)代码切片

本文根据污点序列和依赖关系进行代码切片操作, 首先按照AST的结构深度遍历所有语句节点, 再对每个节点的节点内部结构进行分析并关注语句间的依赖关系, 具体代码切片流程如算法1所示。

为更清晰地阐述代码切片算法的具体过程, 本文以图3为例, 逐步演示如何将源代码转换为CPG, 并在该图上进行遍历分析, 最终提取出目标代码切片。

2.3.2 序列特征提取

代码切片序列中包含复杂的长距离依赖, 仅使用CodeBERT的上下文嵌入可能不足。因此, 本文引入BiLSTM来进一步强化序列特征。具体而言, 代码切片首先通过CodeBERT进行分词和嵌入, 获取每个标记(Token)的特征向量。随后, 该向量序列被送入BiLSTM进行训练, BiLSTM的输出将被用于后续的特征融合, 以更有效地捕捉代码中的长距离依赖关系。

2.4 特征融合

本文通过一个门控机制来实现图特征和文本特征融合, 其核心是使用了门控循环单元(Gated

算法 1 代码切片算法

输入: 抽象语法树AST, 数据依赖图DDG, 控制依赖图CDG, 初始污点变量集合 T_0
 输出: 污染语句集合 S

```

1:  $T \leftarrow T_0$  // 初始化污点序列
2:  $S \leftarrow \emptyset$  // 初始化污染语句集合
3: for each 语句  $s \in \text{AST}$  (深度优先遍历) do
4:   if  $s$ 含外部输入then
5:     将输入变量加入 $T$ , 并令 $S \leftarrow S \cup \{s\}$ 
6:   else if  $s$ 依赖于 $T$ 中变量then //数据依赖传播
7:     if  $s$ 为赋值语句  $x = f(y)$  then
8:       if  $y \in T, x \notin T$  then  $T \leftarrow T \cup \{x\}, S \leftarrow S \cup \{s\}$ 
9:       else if  $x \in T, y \notin T$  then  $T \leftarrow T - \{x\}$  // 消毒
10:    else if  $s$ 为函数调用  $z = f(x_1, \dots, x_n)$  then
11:      if  $\exists x_i \in T$  then  $T \leftarrow T \cup \{z\}$ 的输出变量,  $S \leftarrow S \cup \{s\}$ 
12:      else  $T \leftarrow T - \{z\}$ 的输出变量 // 消毒
13:    else  $S \leftarrow S \cup \{s\}$ 
14:   end if
15:   if  $s \in S$  then // 控制依赖传播
16:     for each  $c \in \text{CDG.control\_dependents}(s)$  do
17:       将 $c$ 中变量加入 $T$ , 并令 $S \leftarrow S \cup \{c\}$ 
18:     end for
19:   end if
20: end for
21: return  $S \neq \emptyset ? S : \text{AST}$ 

```

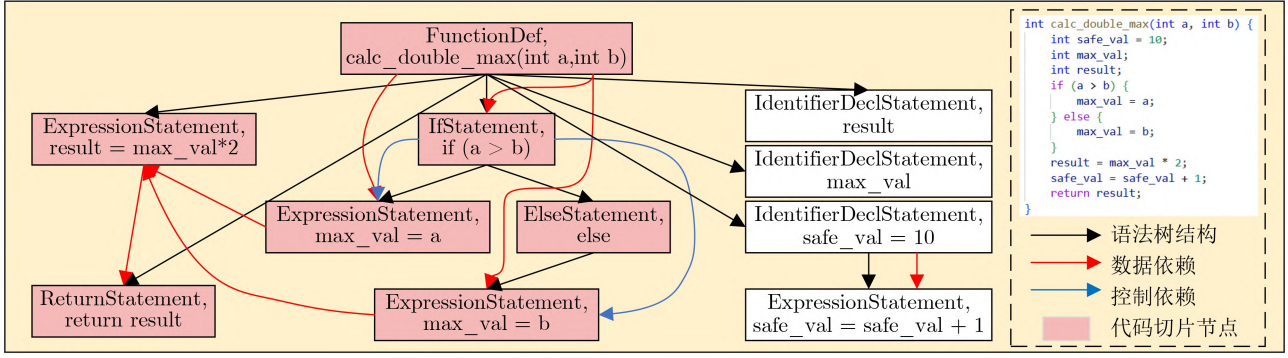


图3 代码切片算法示例

Recurrent Unit, GRU), 它是一种常用的循环神经网络变种。在特征融合中, GRU帮助模型学习如何有效地结合图特征和序列特征。首先, 由于输出的图特征的维度为64, 序列特征的维度为512, 因此特征融合模型首先通过一个线性层将其转换为相同维度的向量以便于后续的GRU使用。然后, 特征融合模型将图特征和序列特征拼接在一起, 如果图特征向量和序列特征向量分别表示为 $\mathbf{g} \in \mathbb{R}^{d_g}$ 和 $\mathbf{t} \in \mathbb{R}^{d_t}$, 其中 d_g 和 d_t 分别表示图特征和序列特征的维度, 则特征拼接操作可以表示为

$$\mathbf{x} = [\mathbf{g}; \mathbf{t}] \quad (3)$$

其中, $\mathbf{x} \in \mathbb{R}^{d_g+d_t}$ 就是拼接后的特征向量, 在本文也可以视其为一个长度为2的序列。

随后, 将转换后的特征向量送入GRU。GRU利用其内部机制更新隐藏状态, 使其天然具备处理序列和融合信息的能力。在本文的特征融合场景中, 尽管输入的并非传统的时间序列, GRU的门控机制依然发挥着关键作用: 它能够根据图特征和序列特征各自的重要性, 动态地调整内部状态, 从而实现两种不同来源特征的有效整合。

2.5 解释器

GNNExplainer^[17]是一种用于解释图神经网络模型决策的工具, 其核心思想是通过最大化模型预测的互信息(Mutual Information, MI)来识别对模型决策最重要的图结构。具体来说, 它试图找到一个子图, 使得这个子图与模型的预测之间的互信息最大化, 其核心思想如公式(4)所示

$$\begin{aligned} & \text{Max}(H(Y|G=G_S)) \\ & = -E_{Y|G_S} \log P(Y=|G=G_S) \end{aligned} \quad (4)$$

对于GNNExplainer, 其主要目标是学习一组边掩码, 并通过重要性排序提取关键子图。然而, 在漏洞检测任务中, 研究者更关注节点的重要性, 因为节点对应具体的代码内容, 更能反映漏洞成因。原有方法通过边的重要性来确定关键子图, 这

种基于边的解释方式在映射到节点层面时容易产生信息损失, 难以准确刻画单个节点对模型预测结果的影响。

因此, 本文针对漏洞检测任务的特殊性, 设计了一个改进型解释器VDExplainer, 其整体流程如图4所示。VDExplainer在GNNExplainer的基础上引入了HITS算法中的权威值与枢纽值的思想, 将节点自身的重要性作为权威值, 将节点与其余高重要性节点连接的能力作为枢纽值, 将边的重要性, 即边掩码作为权重不断迭代更新节点的权威值与枢纽值并在每次迭代之后对其进行归一化处理, 直至两次迭代之间的数值基本不发生变化。具体的计算如公式(5)和公式(6)所示

$$A_i(k+1) = \sum_{j|(j,i) \in E} W \cdot H_j(k) \quad (5)$$

$$H_i(k+1) = \sum_{j|(i,j) \in E} W \cdot A_j(k) \quad (6)$$

其中, $A_i(k)$ 代表第 k 次更新后的节点权威值, $H_i(k)$ 代表第 k 次更新后的节点枢纽值, W 为权重, 初始权威值和枢纽值均为1。

在获取到各个节点的权威值与枢纽值后, 本文将两者相加并按照从大到小的顺序完成节点的排序, 以此作为模型的可解释性结果, 丰富模型的输出并为后续的漏洞定位提供了指导依据。

3 实验结果与分析

3.1 实验设置

本实验基于PyTorch2.1.2实现, 操作系统为CentOS Linux release 8.4.2105, 硬件环境为Intel(R) Xeon(R) Platinum 8358P的CPU、一块

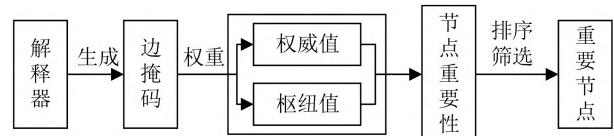


图4 重要节点选择过程

NVIDIA A800 80GB的GPU、156GB内存。经大量实验测试,模型的超参数设置为:池化层筛选比例ratio为0.6, AdamW优化器的学习率lr为0.000 1, β_1 为0.9, β_2 为0.999, 权重衰减weight_decay为0.000 1, batch_size为64。模型为图向量和序列向量分别设置了2个全连接层,其中输入维度分别为64和512,输出维度为256。模型训练周期设置为200轮,并使用了早停法防止模型出现过拟合现象,早停法的性能指标为准确率,patiance为10。采用Accuracy, Precision, Recall和F1-Score作为模型的漏洞检测评价指标。

本研究采用漏洞检测数据集Devign (FFmpeg+Qemu)进行模型训练与评估。Devign由Zhou等人^[6]提出,是一个专注于C/C++语言的权威基准,其样本均源自FFmpeg和QEMU两大开源项目的真实漏洞修复提交。该数据集的一个突出特点在于其漏洞类型的多样性,不仅局限于单一模式,其样本覆盖了多种常见且危害严重的安全缺陷。其中总体数据个数为27 318,包含12 460个有漏洞的样本和14 858个无漏洞的样本。实验将Devign数据集划分为80%训练集、10%验证集和10%测试集。实验数据分布如表1所示。

从表1中可以看出,训练集、验证集和测试集在各项指标上分布相似,保证了评估的公平性。

同时为了验证模型的漏洞检测定位效果,选取Big-Vul数据集进行实验。Big-Vul由Fan等人^[18]提出,其通过爬取CVE数据库和相关源代码仓库中的GitHub链接,收集了涵盖2002年至2019年期间的CVE条目,每个条目由21个特征组成。由于其从已提交的版本补丁中挖掘代码更改信息,以此来定位漏洞文件中哪些代码行发生了修改,因此能够辅助验证模型漏洞定位的有效性。该数据集中拥有11 834个有漏洞样本和253 096个无漏洞样本,数据分布极不均匀,包含漏洞函数的数据仅占4.5%,与此同时,由于本文的漏洞解释定位实验要求数据集必须给出漏洞定位数据,且如果数据本身不包含漏洞或检测模型未检测出数据包含漏洞,该数据对于漏洞解释定位任务的研究均是没有意义的。因此,本文仅选取了Big-Vul数据集中的部分数据进

表1 数据集统计特征分布

特征指标	训练集	验证集	测试集
样本总数	21 854	2 732	2 732
平均函数长度	51.71	49.84	51.98
平均语句数	41.84	40.80	41.81
平均AST深度	13.26	13.27	13.23

行漏洞解释定位的研究。数据选取的要求包括:(1)数据本身必须包含漏洞。(2)必须有对应的修改补丁,且修复方式为删除或更改代码。(3)数据必须被本文模型成功预测为包含漏洞。

3.2 对比实验

为了测试本模型GSVD在漏洞检测任务中的性能,分别与静态漏洞检测工具Cppcheck^[19], Flaw-Finder^[20]和深度学习漏洞检测模型BiLSTM^[21], TextCNN^[22], RoBERTa^[23], CodeBERT^[24], Devign^[6], ReGVD^[8]以及Zeng等人^[25]的模型进行了比较。本文提出的GSVD漏洞检测模型与其余检测工具和深度学习模型在Devign数据集上的结果如表2所示。

由实验结果可以看出,本文提出的GSVD模型不仅改进了现有模型使用的图神经网络结构,还同样使用了基于AST, CDG和DDG的图结构建立方法,能够真正地获取到代码的语法结构。与此同时,GSVD同样考虑到了代码序列信息的重要性,使用当前基于序列的对比模型中性能最优的Code-BERT进行数据预训练,并融入可降低检测粒度、剔除代码噪声的切片算法,最终通过特征融合模型完成漏洞分类检测,性能优于现有漏洞检测工具与模型。从上述实验结果可知,GSVD的准确率和F1得分均达到了对比模型中最高的64.57%和61.89%,召回率也得到了进一步的提升,达到了62.63%,证明其能够有效第完成漏洞检测任务并更好地解决漏洞漏报的问题。

为了验证GSVD中特征融合的设计对漏洞检测任务的正向影响,如表3所示,本文设计实现了特

表2 GSVD实验结果(%)

	Accuracy	Precision	Recall	F1-Score
Cppcheck	57.96	79.55	11.34	19.85
FlawFinder	52.04	46.69	34.41	39.62
BiLSTM	59.37	/	/	/
TextCNN	60.69	/	/	/
RoBERTa	61.05	/	/	/
CodeBERT	62.08	/	/	/
Devign	59.22	57.23	44.46	50.04
ReGVD_GCN	61.90	62.62	42.31	50.50
ReGVD_GGCN	62.12	61.58	46.61	53.06
Zeng's	<u>64.49</u>	<u>64.29</u>	<u>51.08</u>	<u>56.93</u>
GSVD	64.57	61.17	62.63	61.89¹⁾

¹⁾本文所有表格中,若存在黑体与下划线,则将各项评价指标中排名第一的数据用加粗字体表明,排名第二的数据用下划线表明。

征拼接、加权求和以及注意力机制3种融合方法，并完成了与GSVD的融合方法的对比实验。

从表3可以看出，相较于特征拼接、加权求和和注意力机制，GSVD模型在特征融合时的效果最好，准确率、召回率与F1得分均达到了最高的64.57%、62.63%和61.89%，精确率也达到了61.17%，仅次于加权求和方法的63.33%。

本文分析其原因如下：特征拼接是无交互的静态融合，将压力完全转移给下游分类器；加权求和是固定的线性融合，无法动态适应样本，且易导致信息抵消；注意力机制虽实现了动态加权，但本质上是“软选择”，未能像RNN那样建模深层依赖。相比之下，GSVD创新性地将多模态特征视为一个时序序列，并利用GRU进行有状态、非对称的信息整合。这种设计允许模型在序列特征形成的上下文中去理解和融合图特征，从而捕捉到更深刻的特征交互模式，因此取得了最优的综合性能。

3.3 消融实验

针对本文提出的切片算法，本文设置了消融实验，重点对比了数据重复率。该指标过高会导致模型过拟合、泛化能力下降。如表4所示，本文算法的重复率(6.42%)显著低于Vuldeepecker^[3]和SySeVR^[5]。本文分析其原因为：Vuldeepecker仅考虑数据依赖和有限的库/API调用规则；SySeVR增加了控制依赖，且其切片规则更宽泛，因此增加了重复度。相比之下，本文算法同时考虑了“污点”传播与“消毒”过程，这种方法能更精准地聚焦于漏洞相关的代码，有效去除了切片中的噪声，因此降低了数据重复率。

本文提出的代码切片算法无须依赖具体漏洞规则，仅关注污点数据即可完成切片。本方法不仅有效降低了数据冗余，还能完成代码去噪，从而保障

模型检测效果。表5将通过使用切片与不使用切片的对比实验来证明本文提出的切片算法对GSVD模型检测效果与检测粒度的影响。

可以看出，切片后的模型尽管在精确率指标上略有下降，但是在准确率、召回率和F1得分中均有提升，分别升高了0.44%、3.19%和1.53%。与此同时，切片算法将数据从函数级转化为代码片段级，有效降低了待检测源代码的数据量。以代码行数为数据量衡量指标，切片后代码数据量降低66.72%，且模型漏洞检测效果未随数据量减少而下降。这表明，切片算法不仅细化了模型检测粒度，还能有效剔除源代码中与漏洞无关的噪声信息，使模型更聚焦于关键漏洞语句。

3.4 解释器VDExplainer相关实验

由于可解释性难以直接量化，本文设计了漏洞定位实验来间接验证VDExplainer相较于GNNExplainer的优越性。实验设置如下：模型在Devign数据集上训练，在包含行级补丁信息的Big-Vul数据集上测试。测试集经过严格筛选：首先，选取Big-Vul中修复方式为“修改”或“删除”代码的8 243个漏洞实例；然后，仅保留其中被GSVD模型成功检出的样本用于本实验。评估标准为：计算解释器排序的Top- K ($K=30\%$, 40% , 50%)重要节点，是否完全包含了补丁确认的真实漏洞语句。若完全包含，则视为解释定位正确。具体结果如表6所示。

根据实验结果，随着解释器筛选的关键节点比例逐渐减小，漏洞定位准确率呈下降趋势。这主要是因为实验要求解释器给出的节点集合必须完全包含真实漏洞节点，对解释器的可解释性和数据质量均提出了较高要求。即使在最大筛选比例50%时，部分样本前50%节点数量仍小于实际漏洞节点数量；当比例降低至30%时，仅有73.60%的数据能够

表 3 特征融合方法对比(%)

	Accuracy	Precision	Recall	F1-Score
特征拼接	63.36	59.75	61.99	60.85
加权求和	63.14	63.33	46.93	53.91
注意力机制	62.55	59.54	57.68	58.60
GSVD	64.57	61.17	62.63	61.89

表 4 数据重复度对比(%)

方法	数据重复率
未切片	0.20
Vuldeepecker	19.58
SySeVR	22.10
Ours	6.42

表 5 代码切片影响

	准确率	精确率	召回率	F1得分	源代码平均行数
未切片	64.13%	61.30%	59.44%	60.36%	51.98
切片	64.57%	61.17%	62.63%	61.89%	17.30

表 6 解释器解释定位实验(%)

解释器	重要节点筛选比例	漏洞定位准确率
GNNExplainer	30	18.37
	40	28.94
	50	41.12
VDExplainer	30	24.30
	40	35.68
	50	48.77

保证前30%的节点数量高于给定的漏洞节点数量。同时, Big-Vul数据集中的漏洞语句主要由补丁修改位置标注, 仅能部分反映漏洞成因, 也限制了解释器性能的发挥。

尽管存在上述限制, 实验结果显示本文提出的VDExplainer相较于GNNExplainer在漏洞定位上仍具优势: 在前30%、40%和50%节点数量下, 定位准确率分别提升 5.93%、6.74% 和 7.65%。这是因为GNNExplainer基于关键边生成子图, 存在节点信息映射过程中的信息损失; 而VDExplainer同时考虑节点的重要性与连接能力, 直接将边掩码映射到节点, 减少中间步骤, 使解释器能够基于节点完成分析, 从而更贴合漏洞检测任务需求并提升解释能力。

4 结束语

本文针对现有漏洞检测技术存在的不足, 提出了一种基于图和代码切片的可解释性漏洞检测方法GSVD。本模型充分利用源代码的AST, DDG和CDG图结构信息, 通过带有自注意力池化层的GGCN提取代码的深层特征。同时, 本文设计了一种新的基于污点分析的代码切片算法, 通过BiLSTM完成对代码切片的序列特征提取, 并通过GRU实现多元特征的融合。最后, 为了解决漏洞检测定位的问题, 针对漏洞检测场景的特殊性, 设计了一个VDExplainer解释器。本文的研究为可解释性漏洞检测提供了一种新的思路, 结合了图结构和代码切片技术, 相较于其他漏洞检测方法体现了一定的优越性。然而, 本研究仍存在可扩展空间: 例如, 将模型扩展至Java, Python等语言, 这需要适配不同语言的语法结构及Joern工具生成的特定代码属性图的差异; 此外, 未来的研究还可探寻二进制分析更多维度的特征, 并优化特征提取与融合策略。

参考文献

- [1] GAO Qing, MA Sen, SHAO Sihao, *et al.* CoBOT: Static C/C++ bug detection in the presence of incomplete code[C]. The 26th Conference on Program Comprehension, Gothenburg, Sweden, 2018: 385–388. doi: [10.1145/3196321.3196367](https://doi.org/10.1145/3196321.3196367).
- [2] ZHANG Yu, HUO Wei, JIAN Kunpeng, *et al.* SRFuzzer: An automatic fuzzing framework for physical SOHO router devices to discover multi-type vulnerabilities[C]. The 35th Annual Computer Security Applications Conference, San Juan, USA, 2019: 544–556. doi: [10.1145/3359789.3359826](https://doi.org/10.1145/3359789.3359826).
- [3] LI Zhen, ZOU Deqing, XU Shouhuai, *et al.* VulDeePecker: A deep learning-based system for vulnerability detection[C]. The 25th Annual Network and Distributed Systems Security Symposium, San Diego, USA, 2018. doi: [10.14722/ndss.2018.23158](https://doi.org/10.14722/ndss.2018.23158).
- [4] ZOU Deqing, WANG Sujuan, XU Shouhuai, *et al.* μ VulDeePecker: A deep learning-based system for multiclass vulnerability detection[J]. *IEEE Transactions on Dependable and Secure Computing*, 2021, 18(5): 2224–2236. doi: [10.1109/TDSC.2019.2942930](https://doi.org/10.1109/TDSC.2019.2942930).
- [5] LI Zhen, ZOU Deqing, XU Shouhuai, *et al.* SySeVR: A framework for using deep learning to detect software vulnerabilities[J]. *IEEE Transactions on Dependable and Secure Computing*, 2022, 19(4): 2244–2258. doi: [10.1109/TDSC.2021.3051525](https://doi.org/10.1109/TDSC.2021.3051525).
- [6] ZHOU Yaqin, LIU Shangqing, SLOW J, *et al.* Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks[C]. The 33rd International Conference on Neural Information Processing Systems, Vancouver, Canada, 2019: 915. doi: [10.5555/3454287.3455202](https://doi.org/10.5555/3454287.3455202).
- [7] FENG Qi, FENG Chendong, and HONG Weijiang. Graph neural network-based vulnerability predication[C]. 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME), Adelaide, Australia, 2020: 800–801. doi: [10.1109/ICSME46990.2020.00096](https://doi.org/10.1109/ICSME46990.2020.00096).
- [8] NGUYEN V A, NGUYEN D Q, NGUYEN V, *et al.* ReGVD: Revisiting graph neural networks for vulnerability detection[C]. The ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings, Pittsburgh, USA, 2022: 178–182. doi: [10.1145/3510454.3516865](https://doi.org/10.1145/3510454.3516865).
- [9] CHAKRABORTY S, KRISHNA R, DING Yangruibo, *et al.* Deep learning based vulnerability detection: Are we there yet?[J]. *IEEE Transactions on Software Engineering*, 2022, 48(9): 3280–3296. doi: [10.1109/TSE.2021.3087402](https://doi.org/10.1109/TSE.2021.3087402).
- [10] ALI G M A and CHEN Hongsong. Contract-guardian: A bagging-based gradient boosting decision tree for detection vulnerability in smart contract[J]. *Cluster Computing*, 2025, 28(8): 528. doi: [10.1007/s10586-025-05230-2](https://doi.org/10.1007/s10586-025-05230-2).
- [11] GUO Daya, ZHU Qihao, YANG Dejian, *et al.* DeepSeek-Coder: When the Large Language Model Meets Programming -- The Rise of Code Intelligence[EB/OL]. <https://arxiv.org/abs/2401.14196>, 2024.
- [12] DeepSeek-AI, ZHU Qihao, GUO Daya, *et al.* DeepSeek-Coder-V2: Breaking the Barrier of Closed-Source Models in Code Intelligence[EB/OL]. <https://arxiv.org/abs/2406.11931>, 2024.
- [13] AGHAEI E, NIU Xi, SHADID W, *et al.* SecureBERT: A domain-specific language model for cybersecurity[C]. 18th International Conference on Security and Privacy in Communication Networks, Kansas, USA, 2022: 39–56. doi: [10.1007/978-3-031-25538-0_3](https://doi.org/10.1007/978-3-031-25538-0_3).
- [14] SUN Yuqiang, WU Daoyuan, XUE Yue, *et al.* LLM4Vuln:

- A Unified Evaluation Framework for Decoupling and Enhancing LLMs' Vulnerability Reasoning[EB/OL]. <https://arxiv.org/abs/2401.16185>, 2025.
- [15] FAR S M T and FEYZI F. Large language models for software vulnerability detection: A guide for researchers on models, methods, techniques, datasets, and metrics[J]. *International Journal of Information Security*, 2025, 24(2): 78. doi: [10.1007/s10207-025-00992-7](https://doi.org/10.1007/s10207-025-00992-7).
- [16] ZHOU Xin, CAO Sicong, SUN Xiaobing, *et al.* Large language model for vulnerability detection and repair: Literature review and the road ahead[J]. *ACM Transactions on Software Engineering and Methodology*, 2025, 34(5): 145. doi: [10.1145/3708522](https://doi.org/10.1145/3708522).
- [17] YING R, BOURGEOIS D, YOU Jiaxuan, *et al.* GNNExplainer: Generating explanations for graph neural networks[C]. The 33rd International Conference on Neural Information Processing Systems, Vancouver, Canada, 2019: 829. doi: [10.5555/3454287.3455116](https://doi.org/10.5555/3454287.3455116).
- [18] FAN Jiahao, LI Yi, WANG Shaohua, *et al.* A C/C++ code vulnerability dataset with code changes and CVE summaries[C]. The 17th International Conference on Mining Software Repositories, Seoul, Korea, 2020: 508–512. doi: [10.1145/3379597.3387501](https://doi.org/10.1145/3379597.3387501).
- [19] D'ABRUZZO PEREIRA J and VIEIRA M. On the use of open-source C/C++ static analysis tools in large projects[C]. 2020 16th European Dependable Computing Conference (EDCC), Munich, Germany, 2020: 97–102. doi: [10.1109/EDCC51268.2020.00025](https://doi.org/10.1109/EDCC51268.2020.00025).
- [20] FERSCHKE O, GUREVYCH I, and RITTBERGER M. FlawFinder: A modular system for predicting quality flaws in wikipedia[C]. CLEF 2012 Evaluation Labs and Workshop, Online Working Notes, Rome, Italy, 2012: 1178.
- [21] GRAVES A. Long short-term memory[M]. GRAVES A. Supervised Sequence Labelling with Recurrent Neural Networks. Berlin: Springer, 2012: 37–45. doi: [10.1007/978-3-642-24797-2_4](https://doi.org/10.1007/978-3-642-24797-2_4).
- [22] CHEN Yahui. Convolutional neural network for sentence classification[D]. [Master dissertation], University of Waterloo, 2015.
- [23] LIU Yinhan, OTT M, GOYAL N, *et al.* RoBERTa: A robustly optimized BERT pretraining approach[C]. International Conference on Learning Representations, Addis Ababa, Ethiopia, 2020.
- [24] FENG Zhangyin, GUO Daya, TANG Duyu, *et al.* CodeBERT: A pre-trained model for programming and natural languages[C]. Findings of the Association for Computational Linguistics: EMNLP 2020, 2020: 1536–1547. doi: [10.18653/v1/2020.findings-emnlp.139](https://doi.org/10.18653/v1/2020.findings-emnlp.139).
- [25] ZENG Ciling, ZHOU Bo, DONG Huoyuan, *et al.* A general source code vulnerability detection method via ensemble of graph neural networks[C]. The 6th International Conference on Frontiers in Cyber Security, Chengdu, China, 2023: 560–574. doi: [10.1007/978-981-99-9331-4_37](https://doi.org/10.1007/978-981-99-9331-4_37).
- 高文超：女，副教授，研究方向为自然语言处理、计算机视觉、大数据。
- 索建华：男，硕士生，研究方向为大模型、自然语言处理。
- 张 傲：男，硕士，研究方向为大数据、人工智能和软件安全。

责任编辑：廖海贝

An Interpretable Vulnerability Detection Method Based on Graph and Code Slicing

GAO Wenchao SUO Jianhua ZHANG Ao

(School of Artificial Intelligence, China University of Mining and Technology-Beijing, Beijing 100083, China)

Abstract:

Objective Deep learning technology is widely applied to source code vulnerability detection. Existing approaches are mainly sequence-based or graph-based. Sequence-based models convert structured code into linear sequences, which causes loss of syntactic and structural information and often results in a high false-positive rate. Graph-based models capture structural features but cannot represent execution order, and their detection granularity is usually limited to the function level. Both types of methods lack interpretability, which limits the ability of developers to locate vulnerability sources. Large Language Models (LLM) show progress in code understanding, yet they still present high computational cost, hallucination risk in security analysis, and insufficient modeling of complex program logic. To address these issues, an interpretable vulnerability detection method based on graphs and code slicing (GSVD) is proposed. The method integrates structural semantics and sequential information and provides fine-grained, line-level explanations for prediction results.

Methods The proposed method contains four modules: code graph feature extraction, code sequence feature extraction, feature fusion, and an interpreter module (Fig. 1). First, the source code is normalized, and the Joern static analysis tool is used to generate multiple code graphs, including the Abstract Syntax Tree (AST), Data Dependency Graph (DDG), and Control Dependency Graph (CDG). These graphs represent program structure, data flow, and control flow. Node features are initialized using CodeBERT embeddings combined with one-hot encodings of node types. With the adjacency matrix of each graph, a Gated Graph Convolutional Network (GGCN) with a self-attention pooling layer is applied to extract deep structural semantic features. A code slicing algorithm based on taint analysis (Algorithm 1) is then designed. In this algorithm, taint sources are identified, and taints are propagated according to data and control dependencies to generate concise slices related to potential vulnerabilities. These slices remove unrelated code and are processed using a Bidirectional Long Short-Term Memory (BiLSTM) network to capture long-range sequential dependencies. After extracting graph and sequence features, a gating mechanism is introduced for feature fusion. The fused feature vectors are passed through a Gated Recurrent Unit (GRU), which learns dependencies between structural and sequential representations through its dynamic state updates. To support both vulnerability detection and localization, a VDExplainer is designed. Inspired by the HITS algorithm, it iteratively computes node “authority” and “hub” scores under an edge-mask constraint to estimate node importance and provide node-level interpretability for vulnerability explanation.

Results and Discussions The effectiveness of the GSVD model is evaluated through comparative experiments on the Devign dataset (FFmpeg + Qemu), as shown in (Table 2). GSVD is benchmarked against several baseline models and achieves the highest accuracy and F1-score, reaching 64.57% and 61.89%, respectively. The recall improves to 62.63%, indicating that the method enhances vulnerability detection and reduces missed reports. To assess the effectiveness of the GRU-based fusion module, three fusion strategies were compared: feature concatenation, weighted sum, and attention mechanism (Table 3). GSVD delivers the best overall performance. Although its precision (61.17%) is slightly lower than the weighted sum method (63.33%), its combined accuracy, recall, and F1-score show more consistent performance. Ablation studies (Tables 4–5) further highlight the contribution of the slicing algorithm. The taint propagation-based slicing method reduces the average number of code lines from 51.98 to 17.30, a 66.72% reduction, and lowers the data redundancy rate to 6.42%. In contrast, VulDeePecker and SySeVR report 19.58% and 22.10%, respectively. This reduction in noise leads to a 1.53% gain in F1-score, confirming that the slicing module enhances focus on critical code segments. The interpretability of GSVD is validated on the Big-Vul dataset using the VDExplainer module (Table 6). Compared with the standard GNNExplainer, the proposed method achieves higher localization accuracy at all evaluation thresholds. When 50% of the nodes are selected, localization accuracy improves by 7.65%, highlighting the advantage of VDExplainer in node-level vulnerability explanation. In summary, GSVD demonstrates strong performance in detection accuracy and interpretability, offering practical support for both vulnerability identification and localization.

Conclusions The GSVD model addresses the limitations of single-modal methods by integrating graph structures with taint-based code slices. It improves both detection accuracy and interpretability. The VDExplainer enables node-level and line-level localization, enhancing the practical applicability of the model. Experimental findings support the advantages of the proposed method in detection performance and interpretability.

Key words: Vulnerability detection; Deep learning; Graph neural network; Code slicing; Interpretability